

Data Engineering PySpark vs Pandas



Data Structure

Distributed DataFrame
(`pyspark.sql.DataFrame`)

```
df = spark.read.csv("data.csv")
```

PySpark

Single-machine DataFrame
(`pandas.DataFrame`)

```
df = pd.read_csv("data.csv")
```

Pandas



Shwetank Singh

GritSetGrow - GSGLearn.com



Data Processing

PySpark

Designed for large-scale data processing across clusters. Supports lazy evaluation and optimizations.

```
df.filter(df['age'] > 30).select('name', 'age').show()
```

Pandas

Designed for smaller datasets that fit into memory. Eager evaluation, less suited for large datasets.

```
df[df['age'] > 30][['name', 'age']]
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Performance

PySpark

Handles large datasets by distributing computations across clusters.

```
df.groupBy("department") |\n.agg({"salary": "avg"}).show()
```

Pandas

Best for in-memory data manipulation, limited by single machine's memory.

```
df.groupby("department")["salary"].mean()
```



Shwetank Singh

GritSetGrow - GSGLearn.com



APIs and Functions

PySpark

Rich set of SQL-like APIs, DataFrame and SQL queries, strong integration with Spark SQL.

```
df.withColumn('salary_increase', df['salary'] *  
1.10).show()
```

Pandas

Comprehensive set of data manipulation tools, including reshaping, merging, and pivoting.

```
df['salary_increase'] = df['salary'] * 1.10
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Scalability

PySpark

Scalable to handle big data across distributed systems.

```
df = spark.read.json("s3a://bucket/data.json")
```

Pandas

Limited by single-node processing and available memory.

```
df = pd.read_json("data.json")
```



Integration

PySpark

Integrates well with Hadoop, Hive, HDFS, and cloud storage systems like S3.

```
df.write.mode("overwrite").parquet("output/path")
```

Pandas

Integrates with various data formats, but less seamless integration with big data tools.

```
df.to_parquet("output/path")
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Memory Management

PySpark

Optimized for out-of-memory datasets, manages memory through Spark's distributed architecture.

```
df = df.cache()
```

Pandas

Requires all data to fit into memory, can cause issues with very large datasets.

N/A



Shwetank Singh
GritSetGrow - GSGLearn.com



Data Handling

PySpark

Can handle structured and semi-structured data using Spark SQL and DataFrame APIs.

```
df = df.withColumn('new_col', df['existing_col'] * 2)
```

Pandas

Primarily designed for structured data, with support for time series and other data types.

```
df['new_col'] = df['existing_col'] * 2
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Lazy Evaluation

PySpark

Yes, transformations are lazily evaluated until an action is called.

```
result = df.select("name").filter(df["age"] > 30)
```

Pandas

No, operations are evaluated immediately.

```
result = df['name'][df['age'] > 30]
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Data Operations

PySpark

Supports complex operations like joins, aggregations, and window functions on distributed datasets.

```
df.groupBy("category").count().show()
```

Pandas

Supports similar operations but not optimized for distributed systems.

```
df.groupby("category").count()
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Fault Tolerance

PySpark

Built-in fault tolerance with lineage information and DAGs.

```
df = spark.read.csv("data.csv", schema=schema,  
mode="DROPMALFORMED")
```

Pandas

No built-in fault tolerance; must handle errors manually.

```
df = pd.read_csv("data.csv", error_bad_lines=False)
```



Streaming Support

PySpark

Supports real-time data processing using Spark Streaming or Structured Streaming.

```
streamingDF = spark.readStream.option("sep",  
";").csv("path/to/files")
```

Pandas

No native support for real-time streaming; limited to batch processing.

N/A



Machine Learning

PySpark

Integrates with MLlib for scalable machine learning.

```
from pyspark.ml.classification import  
LogisticRegression; lr = LogisticRegression(maxIter=10)
```

Pandas

Works with libraries like scikit-learn for machine learning on smaller datasets.

```
from sklearn.linear_model import LogisticRegression;  
lr = LogisticRegression(max_iter=100)
```



Deployment

PySpark

Typically deployed on distributed clusters (YARN, Mesos, Kubernetes).

```
df.write.mode("overwrite").json("output/path")
```

Pandas

Typically used on single machines, but can be scaled using Dask or distributed computing frameworks.

```
df.to_json("output/path")
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Joins

PySpark

Supports broadcast joins for small datasets, optimizing performance by broadcasting the smaller DataFrame to all worker nodes.

```
df1.join(df2, df1.id == df2.id, "inner").show()
```

Pandas

Joins are performed on single-node, and performance may degrade with very large datasets.

```
pd.merge(df1, df2, on='id', how='inner')
```



Window Functions

PySpark

Provides advanced window functions for operations like ranking, cumulative sums, and moving averages.

```
from pyspark.sql.window import Window;  
df.withColumn("rank", rank() |  
.over(Window.partitionBy("dept") |  
.orderBy("salary"))).show()
```

Pandas

Similar functions available but limited to the memory of a single machine.

```
df['rank'] = df.groupby('dept')  
['salary'].rank(method='first')
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Handling Missing Data

PySpark

Offers various functions for handling missing data, including `dropna()` and `fillna()`.

```
df.dropna().show()
```

Pandas

Similar functionality with `dropna()` and `fillna()` but limited to in-memory datasets.

```
df.dropna()
```



Data Partitioning

PySpark

Automatically partitions data for distributed processing.

```
df = df.repartition(5)
```

Pandas

No automatic partitioning, relies on single-machine memory.

N/A



Shwetank Singh
GritSetGrow - GSGLearn.com



Serialization

PySpark

Supports different serialization formats like Avro, Parquet, and ORC for efficient data storage and transfer.

```
df.write.format("parquet").save("output/path")
```

Pandas

Supports JSON, CSV, and pickle for data serialization.

```
df.to_pickle("output.pkl")
```



Shwetank Singh

GritSetGrow - GSGLearn.com



UDFs (User Defined Functions)

PySpark

Allows the creation of UDFs for custom operations on DataFrames.

```
from pyspark.sql.functions import udf;  
@udf def add_one(x):  
    return x + 1;  
df.withColumn("new_col", add_one(df["col"])).show()
```

Pandas

Supports lambda functions and apply() for custom operations.

```
df['new_col'] = df['col'].apply(lambda x: x + 1)
```



Date and Time Functions

PySpark

Rich support for date and time functions, including conversion, formatting, and arithmetic operations.

```
df.withColumn("day",  
dayofmonth(df["date_col"])).show()
```

Pandas

Extensive support for datetime operations with `pd.to_datetime` and `pd.DateOffset`.

```
df['day'] = df['date_col'].dt.day
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Caching

PySpark

Can cache datasets in memory across distributed nodes.

`df.cache()`

Pandas

Caching is limited to the available memory on a single machine.

`df.copy()`



Aggregation

PySpark

Supports advanced aggregation functions, including groupBy and agg.

```
df.groupBy("department") |\n.agg({"salary": "avg"}).show()
```

Pandas

Similar support but limited to in-memory processing.

```
df.groupby("department")["salary"].agg('mean')
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Integration with SQL

PySpark

Allows seamless integration with SQL queries using `spark.sql()` method.

```
spark.sql("SELECT * FROM table WHERE age > 30").show()
```

Pandas

Limited SQL-like operations using `query()` method.

```
df.query("age > 30")
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Machine Learning Pipelines

PySpark

Integrated with MLlib for creating machine learning pipelines.

```
from pyspark.ml import Pipeline;  
pipeline = Pipeline(stages=[stage1, stage2])
```

Pandas

Can integrate with scikit-learn pipelines for machine learning.

```
from sklearn.pipeline import Pipeline;  
pipeline = Pipeline(steps=[('stage1', stage1), ('stage2',  
stage2)])
```



Graph Processing

PySpark

PySpark GraphFrames library for graph processing and analytics.

```
from graphframes import GraphFrame; gf =  
GraphFrame(vertices, edges); gf.vertices.show()
```

Pandas

No native support; requires additional libraries like NetworkX.

```
import networkx as nx; G = nx.Graph();  
G.add_nodes_from([(1, {"label": "A"}), (2, {"label":  
"B"})])
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Text Processing

PySpark

Can process large-scale text data using distributed systems like Spark MLlib.

```
df.select(explode(split(df['text'], ' ')) \
.alias('word')).groupBy('word').count().show()
```

Pandas

Good for small to medium-sized text data processing using in-built Pandas functions.

```
df['word_count'] = df['text'].str.split().apply(len)
```



Data Merging

PySpark

Supports merging datasets using different join strategies and handling skewed data.

```
df1.join(df2, df1.id == df2.id, "outer").show()
```

Pandas

Data merging is straightforward but limited to in-memory operations.

```
pd.concat([df1, df2], axis=1, join='outer')
```



Handling Nested Data

PySpark

Efficiently handles nested data structures like JSON or nested columns in Parquet.

```
df.select(df['nested_col']['inner_col']).show()
```

Pandas

Can handle nested data using `json_normalize` or similar functions.

```
pd.json_normalize(df['nested_col'].tolist())
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Resource Management

PySpark

Supports resource allocation and management through cluster configuration and tuning.

```
df = df.repartition(sc.defaultParallelism)
```

Pandas

No direct resource management; relies on underlying hardware.

N/A



Shwetank Singh

GritSetGrow - GSGLearn.com



Visualization

PySpark

Limited to simple visualizations using built-in functions; can integrate with Matplotlib or Plotly.

```
df.groupBy("category").count().show()
```

Pandas

Extensive visualization capabilities with built-in plotting and integration with libraries like Matplotlib and Seaborn.

```
df['category'].value_counts().plot(kind='bar')
```



Shwetank Singh

GritSetGrow - GSGLearn.com



Data Transformation

PySpark

Can transform data using SQL, DataFrame API, or RDD transformations.

```
df.selectExpr("id", "cast(salary as int) salary").show()
```

Pandas

Primarily uses DataFrame transformations and Pandas functions.

```
df['salary'] = df['salary'].astype(int)
```



Column Operations

PySpark

Rich support for columnar operations, including column expressions and transformations.

```
df.withColumn('new_col', df['existing_col'] * 2).show()
```

Pandas

Comprehensive column operations, but limited by in-memory processing.

```
df['new_col'] = df['existing_col'] * 2
```



Session Management

PySpark

SparkSession manages the Spark application environment, including configuration and context.

```
from pyspark.sql import SparkSession;  
spark =  
SparkSession.builder.appName("example").getOrCreate()
```

Pandas

No session management; operations are performed directly on DataFrames.

N/A



Shwetank Singh

GritSetGrow - GSGLearn.com



THANK
YOU